

Part A - Your Work

What aspects of this project were you specifically responsible for?

I led the creation of the derived classes with responsibility for the unique items and logic therein.

Which classes, functions, or files did you contribute to?

I solely wrote the Weapon, Armor, and Potion classes. Specifically, I created the overrides for `displayDetails()`, `useItem()`, and `typeTag()` in these files.

What decisions did you make to advance the project?

I helped design the way that each item would differ in game mechanics. I designed Weapons with damage and durability, Armor with defense and durability, and Potions with heal value and a one-use boolean.

Part B - OOP Understanding

4. Why did you choose a class-based architecture as opposed to implementing everything in a single file?

It was able to divide responsibilities. Due to the design in classes, I could focus purely on how armor behaves without concerning myself with how the manager loops through the menu.

5. How did your group implement encapsulation in the code?

We made sure that the class members, such as integers representing damage and durability, were inaccessible to anything besides their own member functions.

6. What purpose did the base class serve in your application?

The inheritance mechanism automatically included common base class fields like `id`, `name`, and `price` into the Weapon, Armor, and Potion classes. This eliminated the need to repeat those data structures in three places.

7. Can you describe polymorphism usage in your project?

In our application, everything was stored as an `Equipment*` pointer. During use in a quest, the manager invoked `useItem()` on a pointer. C++ then determined which `useItem()` implementation to call according to its virtual table entry.

8. Why was having a base class important in this application?

Without a base class, our manager would have needed to create three independent inventories (one for each item type), three search functions for finding items, and three save functions for each type.

9. Which operators were overloaded and why?

We overloaded the == operator to enable comparison by item IDs, the << operator to print details of an item, and the + operator for combining rental fees in a RentalRecord instance.

10. What benefit did the template provide?

We used a generic attribute searchByAttribute template. This enabled writing the search loop only once while applying it to searching the inventory for matching items or active rentals for matching tickets.

Part C - Technical Learning

11. Describe the process of file handling in your application.

In our project, everything was saved in a single CSV file called guild_save.csv. The FileHandler class read each line, determined whether it contained an item or a rental, and dynamically loaded corresponding objects.

12. Was stringstream used in the process?

We used std::stringstream to read CSVs. When loading the inventory, we pushed each line into a stringstream and parsed it field-by-field with a comma delimiter using getline().

13. What exception cases did your application cover?

We covered InvalidItemID for typing a bad ID, FileAccessErrorException when loading a corrupt or missing save file, and NegativeGoldException for trying to spend too much.

14. What bug did you encounter during development and how was it addressed?

We encountered a severe bug where a comma character inside an item description would cause our CSV splitter to produce 10 fields rather than 9. We resolved it with escapeField(), which converted commas to \; before saving and restoring them when loading.

15. What insights did you gain regarding organization of multi-file projects?

We learned that splitting header and .cpp declaration properly is very important. Separating the ownership to my derived item files and teammates' base and manager files was crucial.

Part D - AI Use and Academic Integrity

16. How was AI used in the course of working on this project?

Our group primarily used Anthropic's Claude. It helped scaffold .h files from the UML diagram, debugged errors in code, and produced nicely formatted documents.

17. In what ways was the AI helpful to your work?

Claude was quite handy for figuring out the symptoms of a bug. When we failed to invoke polymorphic function, it helped us trace down the issue. It was also helpful when generating boilerplate content, such as ANSI color codes chart.

18. Were there instances in which the AI gave advice that turned out to be incorrect?

Several times. Initially, it insisted on `dynamic_cast` for the polymorphism bug, despite the fact that we just forgot to declare virtual functions. It also suggested to use `std::shared_ptr` and `std::regex`, both of which we refused since the rubric specified no.

19. How did you ensure that AI's code or explanations are indeed correct?

Everything we received from AI was double-checked. Code suggestions were immediately compiled and tested. If none of us could properly explain why something worked, we scrapped it.

20. Explain in your own words how AI helped with learning rather than taking your place?

AI served as a research assistant. We controlled the flow of logic. For example, when AI suggested some probabilities for random events, we rebalanced everything manually instead of accepting its figures.

Part E - Reflection

21. What do you consider to be the most challenging aspect of this project?

The challenge was in making the random events connect with our inherited classes. To allow the `GuildManager` to generate combat or defense events via a base pointer, I spent considerable time experimenting with solutions.

22. What do you believe to be the most valuable lesson that you've learned throughout the project?

Being able to see polymorphism in action. The moment when I realized how a vtable lookup enables C++ to select the correct `useItem()` method was quite enlightening.

23. If you had more time, what would you change about the project?

I would enrich the game economy. While currently designed as a simple proof-of-concept, adding more derived classes (Magical Artifacts, for example) or making the economy more complex would be interesting.

24. What achievement in particular are you most proud of in this project?

I am particularly proud that our OOP design did not feel like an afterthought to tick off the assignment. Each of the classes I created served a purpose of solving a certain design issue.

25. How did this capstone project help you understand OOP better?

It made me think structurally before programming. Designing the Equipment inheritance hierarchy first helped me learn how inheritance avoids code duplication and makes the system easily expandable.